01

Ruta y preparación

Una aplicación guarda registros fechados. El recorrido consiste en **preparar espacio, crear objetos, observar su almacenamiento, moverlos y organizar los registros por particiones**. ALUMNO es una cuenta local ya existente; todos los ejemplos pertenecen a la misma PDB y usan nombres nuevos.

Las operaciones DDL requieren autorización, cuota y espacio; pueden confirmar cambios pendientes y no se deshacen con ROLLBACK. La cuenta administradora prepara los tablespaces; ALUMNO crea los objetos. No se crean tablas de aplicación en SYS ni SYSTEM.

1. Comprobar el destino y la configuración OMF

ADMINISTRACIÓN · EN LA PDB · SOLO LECTURA

```
SELECT SYS_CONTEXT('USERENV','SESSION_USER') AS usuario,
       SYS_CONTEXT('USERENV','CON_NAME') AS pdb
FROM dual;
SELECT value FROM v$parameter
WHERE name = 'db_create_file_dest';
```

Los ejemplos de archivos sin nombre requieren DB_CREATE_FILE_DEST configurado: Oracle Managed Files (OMF) genera sus nombres. Si el valor está vacío, DATAFILE debe incluir una ruta de archivo nueva y autorizada en el servidor Oracle.

2. Crear tres destinos y conceder espacio a ALUMNO

ADMINISTRACIÓN · DDL

```
CREATE SMALLFILE TABLESPACE ts_demo_datos
DATAFILE  'ts_demo_datos' SIZE 32M AUTOEXTEND ON NEXT 8M
MAXSIZE 128M   EXTENT MANAGEMENT LOCAL AUTOALLOCATE
SEGMENT SPACE MANAGEMENT AUTO;
CREATE SMALLFILE TABLESPACE ts_demo_indices
DATAFILE 'ts_demo_indices' SIZE 32M AUTOEXTEND ON NEXT 8M
MAXSIZE 128M   EXTENT MANAGEMENT LOCAL AUTOALLOCATE
SEGMENT SPACE MANAGEMENT AUTO;
CREATE SMALLFILE TABLESPACE ts_demo_hist
DATAFILE 'ts_demo_hist'  SIZE 32M AUTOEXTEND ON NEXT 8M
MAXSIZE 128M   EXTENT MANAGEMENT LOCAL AUTOALLOCATE
SEGMENT SPACE MANAGEMENT AUTO;
GRANT CREATE TABLE TO alumno;
ALTER USER alumno QUOTA 64M ON ts_demo_datos;
ALTER USER alumno QUOTA 64M ON ts_demo_indices;
ALTER USER alumno QUOTA 64M ON ts_demo_hist;
```

LMT gestiona extensiones; ASSM, bloques disponibles dentro de los segmentos; AUTOEXTEND, el crecimiento del archivo. La cuota de 64M limita la asignación a ALUMNO, no reserva disco.

3. Añadir capacidad y consultar los archivos

ADMINISTRACIÓN · DDL Y CONSULTA

```
ALTER TABLESPACE ts_demo_datos
  ADD DATAFILE SIZE 16M AUTOEXTEND ON NEXT 8M MAXSIZE 64M;
SELECT tablespace_name, file_name, bytes, autoextensible
FROM dba_data_files
WHERE tablespace_name LIKE 'TS_DEMO_%';
```

ADD DATAFILE añade otro archivo porque el tablespace es SMALLFILE. Aumenta su capacidad; no añade filas ni extensiones a una tabla por sí solo.

SQL · CREATE TABLESPACE SQL · ALTER TABLESPACE SQL · ALTER USER

02

Crear y observar los objetos

Con ALUMNO, se crea SQL_REGISTROS en TS_DEMO_DATOS y un índice de fechas en TS_DEMO_INDICES. La definición, el propietario y el almacenamiento son aspectos distintos.

4. Crear la tabla, insertar dos registros y crear el índice

ALUMNO · DDL, DATOS Y CONSULTA

```
CREATE TABLE sql_registros (
  id NUMBER, fecha DATE, detalle VARCHAR2(80)
) SEGMENT CREATION IMMEDIATE
  PCTFREE 20 TABLESPACE ts_demo_datos;
INSERT INTO sql_registros
  VALUES (1, DATE '2025-12-31', 'Registro anterior');
INSERT INTO sql_registros
  VALUES (2, DATE '2026-01-01', 'Registro reciente');
COMMIT;
CREATE INDEX ix_sql_fecha ON sql_registros (fecha)
  TABLESPACE ts_demo_indices;
SELECT id, fecha, detalle FROM sql_registros ORDER BY id;
```

IMMEDIATE solicita crear el segmento al crear la tabla; no hay una sentencia CREATE SEGMENT independiente. PCTFREE 20 reserva margen para actualizaciones, no el 20 % del tablespace. El índice tiene su propio segmento.

5. Comprobar definición, ubicación y asignación

ALUMNO · SOLO LECTURA

```
SELECT table_name, tablespace_name, segment_created, pct_free
FROM user_tables WHERE table_name = 'SQL_REGISTROS';
SELECT index_name, table_name, tablespace_name, status
FROM user_indexes WHERE index_name = 'IX_SQL_FECHA';
SELECT segment_name, segment_type, tablespace_name,
       bytes, blocks, extents
FROM user_segments
WHERE segment_name IN ('SQL_REGISTROS','IX_SQL_FECHA');
SELECT tablespace_name, bytes, max_bytes FROM user_ts_quotas
WHERE tablespace_name LIKE 'TS_DEMO_%';
```

USER_TABLES y USER_INDEXES describen los objetos; USER_SEGMENTS muestra espacio asignado. BYTES no mide el tamaño exacto de los valores de las filas; BLOCKS no cuenta filas.

6. Consultar las extensiones y el tamaño de bloque

ALUMNO · SOLO LECTURA

```
SELECT segment_name, extent_id, blocks, bytes
FROM user_extents
WHERE segment_name = 'SQL_REGISTROS'
ORDER BY extent_id;
SELECT tablespace_name, contents, block_size,
       extent_management, segment_space_management
FROM user_tablespaces
WHERE tablespace_name LIKE 'TS_DEMO_%';
```

Cada fila de USER_EXTENTS representa una extensión. Una extensión agrupa bloques consecutivos de un archivo. BLOCK_SIZE se expresa en bytes; los bloques incluyen datos y estructuras de gestión.

SQL · CREATE TABLE SQL · CREATE INDEX Reference · USER_SEGMENTS

03

De los bloques al traslado

7. Localizar extensiones en archivos

ADMINISTRACIÓN · EN LA MISMA PDB · SOLO LECTURA

```
SELECT extent_id, file_id, relative_fno, block_id, blocks,
       block_id + blocks - 1 AS bloque_final
FROM dba_extents
WHERE owner = 'ALUMNO' AND segment_name = 'SQL_REGISTROS'
ORDER BY extent_id;
```

El rango termina en BLOCK_ID + BLOCKS - 1. USER_EXTENTS no incluye FILE_ID ni BLOCK_ID. Las extensiones pueden estar en los dos archivos de TS_DEMO_DATOS; una extensión no cruza de archivo.

8. Identificar el bloque de una fila

ALUMNO · SOLO LECTURA

```
SELECT id, ROWID AS direccion,
       DBMS_ROWID.ROWID_RELATIVE_FNO(ROWID) AS archivo_rel,
       DBMS_ROWID.ROWID_BLOCK_NUMBER(ROWID) AS bloque,
       DBMS_ROWID.ROWID_ROW_NUMBER(ROWID) AS posicion
FROM sql_registros ORDER BY id;
```

ARCHIVO_REL corresponde al número relativo, no necesariamente a FILE_ID. ROWID no es una clave de negocio y puede cambiar al reorganizar la tabla. No se crean ni se mueven bloques aislados con CREATE BLOCK o MOVE BLOCK.

9. Reservar una extensión y trasladar los objetos

ALUMNO · DDL

```
ALTER TABLE sql_registros ALLOCATE EXTENT;
ALTER TABLE sql_registros
  MOVE TABLESPACE ts_demo_hist UPDATE INDEXES;
ALTER INDEX ix_sql_fecha
  REBUILD TABLESPACE ts_demo_hist;
```

ALLOCATE EXTENT añade espacio al segmento sin insertar filas; normalmente Oracle lo solicita automáticamente. MOVE reubica la tabla. UPDATE INDEXES mantiene actualizado el índice, pero no lo traslada por sí solo. REBUILD lo reconstruye en el destino indicado.

10. Comprobar el nuevo destino

ALUMNO · SOLO LECTURA

```
SELECT segment_name, segment_type, tablespace_name, bytes
FROM user_segments
WHERE segment_name IN ('SQL_REGISTROS','IX_SQL_FECHA');
SELECT index_name, tablespace_name, status
FROM user_indexes WHERE index_name = 'IX_SQL_FECHA';
```

Ambos segmentos quedan en TS_DEMO_HIST y el índice debe estar VALID. Las filas y el propietario siguen siendo los mismos. Los segmentos LOB, si existieran, requerirían cláusulas específicas; no se trasladan automáticamente con la tabla.

Reference · DBA_EXTENTS PL/SQL · DBMS_ROWID SQL · ALTER TABLE SQL · ALTER INDEX

04

Separar y mover particiones

La versión SQL_REGISTROS_PART mantiene el mismo conjunto de datos, pero divide los registros por fecha. Primero se aloja en TS_DEMO_DATOS; después se mueve únicamente la parte anterior a 2026.

11. Crear las particiones y un índice local

ALUMNO · DDL Y DATOS

```
CREATE TABLE sql_registros_part (
  id NUMBER, fecha DATE NOT NULL, detalle VARCHAR2(80)
) PARTITION BY RANGE (fecha) (
  PARTITION p_anteriores VALUES LESS THAN (DATE '2026-01-01')
    TABLESPACE ts_demo_datos,
  PARTITION p_recientes VALUES LESS THAN (MAXVALUE)
    TABLESPACE ts_demo_datos
);
INSERT INTO sql_registros_part
  SELECT id, fecha, detalle FROM sql_registros;
COMMIT;
CREATE INDEX ix_sql_part_fecha ON sql_registros_part (fecha)
  LOCAL (
    PARTITION ip_anteriores TABLESPACE ts_demo_indices,
    PARTITION ip_recientes TABLESPACE ts_demo_indices
  );
```

La fecha 2026-01-01 pertenece a P_RECIENTES. En este particionado simple, cada partición con segmento creado tiene su propia asignación; el índice local también se divide en particiones.

12. Trasladar la partición histórica y su índice

ALUMNO · DDL

```
ALTER TABLE sql_registros_part MOVE PARTITION p_anteriores
  TABLESPACE ts_demo_hist UPDATE INDEXES;
ALTER INDEX ix_sql_part_fecha REBUILD PARTITION ip_anteriores
  TABLESPACE ts_demo_hist;
```

MOVE PARTITION conserva los límites y las filas de la partición. P_RECIENTES no cambia de destino. UPDATE INDEXES mantiene los índices afectados; la sentencia REBUILD siguiente elige expresamente el destino de IP_ANTERIORES.

13. Comprobar ambas ubicaciones y el estado del índice

ALUMNO · SOLO LECTURA

```
SELECT partition_name, tablespace_name
FROM user_tab_partitions
WHERE table_name = 'SQL_REGISTROS_PART'
ORDER BY partition_position;
SELECT partition_name, tablespace_name, status
FROM user_ind_partitions
WHERE index_name = 'IX_SQL_PART_FECHA'
ORDER BY partition_position;
```

P_ANTERIORES e IP_ANTERIORES quedan en TS_DEMO_HIST; P_RECIENTES sigue en TS_DEMO_DATOS e IP_RECIENTES, en TS_DEMO_INDICES. Las particiones del índice deben estar USABLE. USER_SEGMENTS y USER_EXTENTS permiten seguir su asignación incluyendo PARTITION_NAME.